

给 agent 窄的业务动作不要给任意查询权限 Grounding Salesforce Agentforce with Neo4j Knowledge Graphs

本篇范围声明：这是 Neo4j 开发者博客的一篇**集成指南**（原发于 Medium，标注阅读时长 **9 分钟**）。它讲怎么把 Neo4j 知识图谱接进 Salesforce Agentforce 作为一个「有依据的动作」。下面的「全文翻译」覆盖正文全部八节；参考资源链接与页脚不译。**它是集成方案不是评测：**有架构、有三条实现路线、有设计判断，但**没有性能数据、没有准确率对比**。

核心内容

Q：这篇要解决什么？ A：Salesforce 里的 AI agent 拿不到「连接起来的上下文」。

作者的说法是：Salesforce 是客户工作发生的地方，Agentforce 把 AI agent 带进这些工作流，**但 agent 回答的质量仍然取决于它能够到的上下文**。

一条客户记录（Account record）能告诉你**你的公司知道这个客户什么**。它可能显示不了这个客户周围**更广的网络**：子公司、供应商、竞争对手、高管、所属行业、近期新闻，以及**那些解释「这个客户为什么在变」的二阶关系**。

Q：一个具体场景是什么？ A：作者举的是一个常见的 Salesforce 工作流：

「我明天要和 Neo4j 开会。给我一份关于这家公司、它的竞争对手、供应商、子公司和近期新闻的简报。」

一个有用的回答不能停在 CRM 摘要上，它需要组合四种上下文：

类型	内容
Salesforce 上下文	客户归属、在谈的商机、近期活动、续约日期、支持工单、内部备注
市场上上下文	竞争对手、供应商、子公司、高管、行业、关联公司
近期动态	提到这家公司 或它业务网络中其他实体 的新闻
推理上下文	一条关系为什么重要 ，以及哪些底层数据支撑这个建议

作者的判断：传统检索能处理文档和片段，但很多业务问题本质上是关系问题——「哪些供应商和这个客户连着？」「哪些子公司有近期新闻？」「哪个竞争对手也活跃在同一个行业？」这些是图形状的问题，它们受益于图遍历而不是单纯的文本搜索。

Q：这篇最有分量的设计判断是什么？ A：不要让 LLM 负责系统的每一部分。

作者把职责拆成五份，每一份归不同的东西管：

谁	负责什么
agent	识别用户的意图
action 边界	定义什么可以被调用
Flow（Salesforce 的编排工具）	用 CRM 记录丰富答案
Neo4j	执行图查询
prompt 模板	控制最终的语气和结构

这样做的收益是可治理：你可以审查有哪些 action 可用、检视 action 背后的那条 Cypher 查询、通过 Salesforce 设置管理凭据、并且明确决定哪些图上的事实该被返回给模型。

紧接着是全文最该记住的一句：

For production code, keep the Cypher parameterized and expose narrow actions that match business tasks. "Get account network briefing" is safer and easier for an agent to use than a generic "run arbitrary Cypher" action. 对生产代码，把 Cypher 参数化，暴露与业务任务对应的窄动作。「取客户网络简报」这样一个动作，比一个通用的「跑任意 Cypher」动作更安全、也更容易让 agent 用好。

Q：三条实现路线分别是什么？ A：作者强调它们不是互相竞争的方案，而是给不同 Salesforce 团队的部署选择：

一、Apex Actions（要控制力时用）

Apex 是 Salesforce 自己的编程语言。示例里用 @InvocableMethod 暴露一个「取 Neo4j 组织洞察」动作：Apex 服务通过 Named Credential 调 Neo4j 的 HTTP Query API，收到 JSON，把图的响应简化，返回给 Agentforce。

适合：要在同一个事务流里把 Neo4j 结果和 Salesforce 记录组合；要加自定义校验、错误处理或响应整形；想让集成贴近现有的 Salesforce 开发实践；要从 Flow、prompt 模板或 Lightning 组件复用同一段逻辑。

二、External Service Actions（要低代码时用）

暴露一个带 OpenAPI schema 的 REST API，把 schema 导入 Salesforce，然后把生成的操作当作 Flow 或 Agentforce 里的一个动作用。

示例的桥接层是一个轻量的 **Node.js 服务**，适合跑在 Cloudflare Workers 这类平台上。

适合：**第一版不想碰 Apex**；想把 Neo4j 适配器放在 Salesforce 之外；想把 API 转换逻辑集中在一个小服务里。

三、MCP（要标准化工具层时用）

Model Context Protocol 正在成为把工具和上下文暴露给 AI agent 的通行做法。随着 Salesforce 对 MCP 的支持成熟，Neo4j 的 MCP server 可以提供一条**基于标准**的路子。

但作者在这里重复了同一条警告：

企业级 agent 应当拿到具体的、受治理的工具，而不是不受限的数据库访问。正确的模式是暴露业务上安全的图操作，带清楚的描述、输入和权限。

时间点是明确的：这条集成方式在 Salesforce 的路线图上，计划 2026 年中正式可用（GA）。

Q：这和「在 Salesforce 里做 RAG」有什么不同？ A：作者的区分是：

检索增强生成常被描述成「找到对的文本，把它给模型」。那有用，但 Salesforce 的工作流常常需要的**不只是文本片段，而是连接起来的运营上下文。**

图能回答的四类问题：**哪些公司通过供应商或子公司关系和这个客户相关？哪些新闻不只提到目标客户、还提到它周边网络里的公司？哪些关系解释了一条建议？在 agent 回答之前，哪些 CRM 记录该和外部市场情报组合起来？**

还有一条收益是可解释性：一个有图支撑的回答能指出它用到了哪些实体和关系。作者说这在企业环境里比一个没有可见依据的不透明摘要有用得多。

背景补充 / 点评

「窄动作而不是任意查询」这条判断，值得从这篇里单独取走

这是全文唯一一条真正的工程判断，而且它可迁移。

它反对的是一个很自然的想法：**既然 LLM 会写 SQL/Cypher，那就给它一个「执行查询」的工具，让它自己组织查询。**这个想法的吸引力在于灵活——一个工具覆盖所有问题。

作者反对它的理由有两层：

第一层是安全：「不受限的数据库访问」意味着 agent 可以查到任何数据，**权限边界不存在了。**而窄动作天然带边界——「取客户网络简报」这个动作只能返回它设计好要返回的那几类事实。

第二层是「更容易让 agent 用好」(easier for an agent to use)，这一层更微妙也更实用：一个语义明确的窄动作，agent 选对它的概率高于让 agent 自己拼一条正确的查询。前者是选择题，后者是生成题。

这和昨天读的那篇 GRA 论文的发现对得上：那篇实测发现工具调用失败率和准确率严格同序（DeepSeek 失败率 <1% 拿前三名，GPT-5 Nano 失败 10.2% 是最不准的），并且结论是「可靠的工具使用是比延长推理更强的瓶颈」。窄动作正是降低工具调用失败率的手段——参数少、语义明确、没得拼错。

「不要让 LLM 负责系统的每一部分」是一个架构原则

那张五方职责表值得记住它的形状：agent 只做意图识别，其余四件事分给四个不同的、可审查的组件。

这个划分的意义在于每一份都是可治理的：action 清单可以审、Cypher 可以看、凭据在 Salesforce 设置里管、返回哪些字段是明确决定的。LLM 只在两端出现——识别意图（入口）和整理措辞（出口），中间的取数和组装全部由确定性的东西完成。

对照我在合规检查方案里的三条路线：方案甲（全文直判）把最多的事交给 LLM，方案丙（入库时读成结构化清单）把最多的事交给确定性流程。这篇的原则支持方案丙那个方向——不是因为丙更省，是因为丙的每一步都可审查。

关于 MCP 的两点事实

一、时间点明确：Salesforce 的 MCP 支持计划 2026 年中 GA。作者在正文里两次提到「随着 Salesforce MCP 支持成熟」，说明写这篇的时候它还不可用。

二、作者对 MCP 的态度是有保留的。他说 MCP 路线「有吸引力，因为它能让 agent 通过一个通用协议发现精选过的图工具」，紧接着就说「同样的告诫仍然适用」，重复了那条「具体的、受治理的工具，不是不受限的数据库访问」。

这个重复是刻意的：MCP 让暴露工具变容易了，而「容易暴露」正是「暴露太多」的诱因。

我认为这篇要打折的地方

它是一篇集成指南，不是一个验证过的方案。全文没有回答：

- 这套东西答得准不准——没有任何准确率、召回率数字
- 图查询返回多少字段合适——作者说「明确决定哪些图上的事实该被返回给模型」，但没给判据
- 窄动作要窄到什么程度——「取客户网络简报」够窄吗？如果用户问的是「只要供应商」，是复用这个动作还是再建一个？

最后一个问题是实践中最难的：动作太窄会导致动作数量爆炸，agent 在几十个动作里选反而更容易选错；动作太宽又回到「任意查询」那一头。作者没有讨论这个取舍。

「Salesforce 拥有工作流，Neo4j 拥有连接的上下文」

作者反复用这个句式概括分工，出现了两次。**这个划分本身是清楚的，但它也暴露了这套方案的适用边界**：它假设你**已经**有一张维护好的公司知识图谱（示例用的是 Neo4j 的演示数据库，含组织、人、地点、行业、文章）。

这张图从哪来、谁维护、怎么保证它是新的，这篇一个字都没说。而这恰恰是同一天读的另一篇（Document Intelligence）在解决的问题——**两篇合起来才是完整的一条链**：Document Intelligence 建图，这篇讲怎么把建好的图接进业务 workflow。

对我的启示

第一条，「窄动作」这条原则可以直接用来审我手上的两个东西。

一是 CSRS 的网关端点。现在文档类入库端点只有两个（`parse` 和 `parse_upload`），检索是 `/gateway/search/retrieve`。按这篇的判据，**retrieve 是一个偏「通用」的动作**——它返回 `top_k` 个最相关的块，调用方要自己判断这些块够不够、是不是答非所问。

如果要给 agent 用，更好的形态是几个窄动作，比如「查这条 claim 在文档里有没有支撑」直接返回「有支撑/冲突/无支撑 + 证据」，而不是返回一堆块让 agent 自己判。**这正是我在合规检查方案里说的「判定层放 CSRS」那个选项。**

二是 prove 的指标查询。现在的形态是编译器把本体编译成 SQL，**这本身就是窄动作**——每个指标是一个明确的东西，不是让 LLM 自己写 SQL。**这个设计按这篇的判据是对的**，值得在选型文档里记一笔。

第二条，「不要让 LLM 负责系统的每一部分」应该写进合规检查方案的架构原则。

我给的三条路线（甲全文直判、乙检索加判定、丙入库时读成表）**差别的实质就是「LLM 负责多少」**：

方案	LLM 负责	确定性流程负责
甲	通读全文 + 抽 claim + 判定	拉全文
乙	抽 claim + 判定	检索
丙	入库时抽维度表 + 判定时对照几条	切分、维度归类、取那个维度的全部条目

丙把最多的事交给了确定性流程，所以它每一步都可审查——维度表可以人工看、可以单独评测、抽错了能定位到哪一条。

这一条比我原来给的理由（「丙能撑住大文档」）更根本。撑住大文档是性能理由，可审查是治理理由，**后者在合规场景里权重更高。**

第三条，「容易暴露正是暴露太多的诱因」这个观察，对 MCP 有普遍意义。

作者对 MCP 的保留态度值得记：**协议让暴露工具变容易，就会有人把整个数据库暴露成一个工具。**这和「有了 ORM 就有人写出 N+1 查询」是同一个形状——**便利本身会诱发误用。**

具体到我这边：本机配了 `plugin:playwright:playwright` 这个 MCP（虽然连不上）。将来如果要给 CSRS 或 prove 配 MCP server，这条告诫适用——暴露的应该是「查某个指标」「校验某条 claim」这种业务动作，不是「执行 SQL」「执行 Cypher」。

文中金句

"An Account record can tell you what your company knows about a customer. It may not show the broader network around that customer." 一条客户记录能告诉你，你的公司知道这个客户什么。它可能显示不了这个客户周围更广的网络。

"The important design choice is that the LLM is not responsible for every part of the system." 重要的设计选择是：LLM 不为系统的每一部分负责。

"For production code, keep the Cypher parameterized and expose narrow actions that match business tasks. 'Get account network briefing' is safer and easier for an agent to use than a generic 'run arbitrary Cypher' action." 对生产代码，把 Cypher 参数化，暴露与业务任务对应的窄动作。「取客户网络简报」比一个通用的「跑任意 Cypher」动作更安全，也更容易让 agent 用好。

"Enterprise agents should receive specific, governed tools, not unrestricted database access." 企业级 agent 应当拿到具体的、受治理的工具，而不是不受限的数据库访问。

"Many business questions are relationship questions. ... These are graph-shaped questions, and they benefit from graph traversal rather than text search alone." 很多业务问题本质上是关系问题。……这些是图形状的问题，它们受益于图遍历，而不是单纯的文本搜索。

全文翻译

开篇：为什么 CRM 的 agent 需要连接的上下文

副标题：连接起来的企业上下文如何让 Salesforce 的 agent 更有用、更可解释、更导向行动。

Salesforce 是客户工作发生的地方。销售为客户会议做准备，服务团队调查工单，营收团队决定下一步聚焦哪里。Agentforce 把 AI agent 带进这些 workflow，但 agent 回答的质量仍然取决于它能够到的上下文。

一条客户记录能告诉你你的公司知道这个客户什么。它可能显示不了这个客户周围更广的网络：子公司、供应商、竞争对手、高管、所属行业、近期新闻，以及那些解释「这个客户为什么在变」的二阶关系。那种连接起来的上下文，正是 Neo4j 知识图谱有用的地方。

本文展示如何把 Neo4j 作为一个有依据的动作（grounded action）提供给 Salesforce Agentforce。示例是一个行业研究 agent，把 Salesforce 的工作流层和 Neo4j 的公司知识图谱结合起来，让用户能要一份战略客户简报，并收到一个扎根于显式关系、而不是通用网页式摘要的回答。

（四种上下文的表格与三个关系型问题的例子见「核心内容」一节。）

Neo4j 给 Agentforce 提供了做这类工作所需的连接数据层。与其让语言模型从松散的文本里推断关系，我们可以把精选过的图查询暴露成 Salesforce 的动作。agent 专注在用户意图和工作流上，Neo4j 返回关于这个业务网络的结构化事实。

Agentforce 在哪里合适

Agentforce 的设计围绕能推理、能选择动作、能与 CRM 系统交互的 agent。在 Salesforce 当前的术语里，子 agent（subagent）处理聚焦的工作领域，动作（action）让这些 agent 检索信息、执行逻辑或调用外部系统。

这个模型和 Neo4j 很合：

- 一个子 agent 可以拥有一个聚焦的工作流，比如公司情报或客户研究
- 一个动作可以暴露一项安全的、精选过的 Neo4j 能力，比如「取关于某公司的洞察」
- Flow 可以在图查询前后编排 Salesforce 原生的丰富化
- prompt 模板可以塑造最终回答，而不必把所有指令都塞进 agent 本身

作者说：等 Salesforce 的原生 MCP 集成到来后这个契合会更好，那件事很快会有，届时会进一步探索。

结果是一个指令轻、可审计、Salesforce 团队熟悉的 agent 架构。管理员管 Flow 和 prompt 行为，开发者管图查询和集成代码，业务用户留在 Salesforce 的体验里。

参考用例：公司情报

Neo4j Labs 的 Salesforce Agentforce 集成指南实现了一个公司洞察工作流。agent 从用户请求里捕获一个组织名，调用一个 Salesforce 动作，返回一份结构化简报。

Neo4j 的演示数据库含有公司和新闻数据，实体包括组织、人、地点、行业和文章。对一个具名公司，图查询能取回若干属性：从组织概况，到关键领导层、商业与竞争格局，再到一组提到这家公司或其网络中实体的近期文章。

That response becomes the factual substrate for the Agentforce answer. The language model does not need to invent a company network.

那个响应成为 **Agentforce** 回答的事实基底。语言模型不需要发明一个公司网络。它收到结构化的图数据，用 prompt 模板把它总结成专业格式。

对销售，最终回答可能变成一份会议简报；对客户成功经理，可能变成一份续约风险摘要；对服务团队，可能在升级前提供围绕战略客户的上下文。**底下的模式是同一个：Salesforce 拥有 workflow，Neo4j 拥有连接的上下文。**

一个 Salesforce 原生的集成模式

（五方职责表见「核心内容」一节。）

This makes the integration easier to govern. You can review which actions are available, inspect the Cypher query behind the action, manage credentials through Salesforce setup, and decide exactly which graph facts should be returned to the model.

这让集成更容易治理。你可以审查有哪些动作可用、检视动作背后的那条 Cypher 查询、通过 Salesforce 设置管理凭据，并且明确决定哪些图上的事实该被返回给模型。

把 Neo4j 接到 Agentforce 的三条路

技术指南描述了三条集成路线。**它们与其说是互相竞争的方案，不如说是给不同 Salesforce 团队的部署选择。**

（三条路线的详细内容见「核心内容」一节。）

关于 MCP 那条，作者写道：**MCP 路线有吸引力，因为它能让一个 agent 通过一个通用协议发现精选过的图工具。同样的告诫仍然适用：企业级 agent 应当拿到具体的、受治理的工具，而不是不受限的数据库访问。正确的模式是暴露业务上安全的图操作，带清楚的描述、输入和权限。**

这条集成方式在 Salesforce 的路线图上，计划 2026 年中正式可用。

为什么这比「在 Salesforce 里做 RAG」更多

Retrieval-augmented generation is often described as finding the right text and giving it to a model. That is useful, but Salesforce workflows often need more than text snippets. They need connected operational context.

检索增强生成常被描述成「找到对的文本，把它给模型」。那有用，但 Salesforce 的工作流常常需要的不只是文本片段，而是连接起来的运营上下文。

（图能回答的四类问题见「核心内容」一节。）

Neo4j 让那些关系变得显式。Agentforce 让它们变得可操作，办法是把答案带进用户已经在其中工作的那个 CRM 工作流里。

This also improves explainability. A graph-backed response can point to the entities and relationships used to produce the answer. That is much more useful in an enterprise setting than an opaque summary with no visible grounding.

这还改进了可解释性。一个有图支撑的回答，能指出它为产出这个答案而用到的实体和关系。在企业环境里，这比一个没有可见依据的不透明摘要有用得多。

先试什么

如果你想试这个集成，从 [Apex 或 External Service](#) 那条路开始：

1. 用 Neo4j Labs 集成指南里的示例「公司洞察」动作
2. 把这个动作加进一个聚焦公司或客户情报的 Agentforce 子 agent
3. 用一个具体的 prompt 测试，比如「给我一份关于 Neo4j 的简报，包括竞争对手、供应商、子公司和近期新闻」
4. 通过 Flow 加上 Salesforce 的丰富化，比如客户负责人、在谈商机、近期工单、或即将到来的续约日期

That last step is where the demo becomes a Salesforce workflow instead of a standalone graph lookup.

最后那一步，才是这个演示从「一次独立的图查询」变成「一个 Salesforce 工作流」的地方。Neo4j 提供连接的业务上下文，Salesforce 提供客户上下文和流程。

超越文本：Salesforce 界面里的图上下文

第一版可以在 Agentforce 的对话里返回一份文字简报，但同一份图数据能支撑更丰富的 Salesforce 体验。

举例来说，一个 [Lightning Web Component](#) 可以在客户页面上渲染一个局部的客户网络：客户在中心，周围是连接的供应商和子公司，相关组织上挂着近期文章。销售可以在开会前看一眼这张图，服务经理可以在升级过程中打开同一个视图，营收负责人可以用它理解客户集中度或生态风险。

This is where the combination becomes especially practical. Agentforce can answer the question conversationally, while Salesforce UI components can preserve the graph as an inspectable business object.

这就是这个组合特别实用的地方。Agentforce 能以对话方式回答问题，而 Salesforce 的界面组件能把这张图保留成一个可检视的业务对象。

参考链接

- [Grounding Salesforce Agentforce With Neo4j Knowledge Graphs](#) — 本文原文 (Neo4j Developer Blog on Medium)
- [Neo4j Labs Salesforce Agentforce 集成指南](#) — 文中反复引用的技术指南与示例动作
- [Neo4j Agent Integrations 仓库](#) — 示例代码
- [Neo4j MCP Server](#) — 第三条路线用到的 MCP 实现
- [Salesforce Agentforce APIs and SDKs](#) — Salesforce 侧的接口文档
- [Model Context Protocol](#) — 协议本身